xv6 Process Lifecycle Tracing

GDB-Based Kernel Debugging of Process Creation, Scheduling, and Termination

Author: Mishank Shah (24BCE2467)

Course: BCSE303P — Operating Systems Laboratory | **Base kernel:** xv6-public (MIT, x86)

Debugger: GDB attached to QEMU's built-in gdbstub (`-s -S`, `tcp::26000`)

1. Objective

Every running program in xv6 is represented by one `struct proc` sitting in a fixed-size global table, `ptable`, and the kernel moves that struct through a small, well-defined state machine — `UNUSED → EMBRYO → RUNNABLE → RUNNING → SLEEPING → ZOMBIE` — as it is created, scheduled, blocked, and eventually reaped. This lab traces that state machine directly at the debugger level: GDB is attached to a QEMU instance running the kernel, breakpoints are placed on every function permitted to change a process's state, and struct-level inspection (`print` / `display` on `struct proc` and on the global `ptable`) is used to watch `init`, its children, and the round-robin scheduler in real time — down to the register values saved and restored at each `swtch()` boundary.

2. Concept

`ptable` (`kernel/proc.c`) is a global array of `NPROC` `struct proc` entries guarded by `ptable.lock` ; every process the kernel knows about, from `init` (PID 1) down to the last shell command, occupies one slot. The fields that matter for lifecycle tracing are:

Field	Type	Role
`state`	enum procstate	one of the six lifecycle states
`pid`	int	unique process id, assigned in `allocproc()`
`parent`	struct proc *	creator of this process; reparented to `initproc` on orphaning
`sz`	uint	size of the process's user memory image, in bytes
`kstack`	char *	base of this process's private kernel stack
`tf`	struct trapframe *	saved user-mode registers, sitting on the kernel stack, used to resume user execution
`context`	struct context *	saved kernel-mode callee-saved registers, used only by `swtch()`
`chan`	void *	the wait channel this process is sleeping on, if `SLEEPING`

Two distinct kinds of "switch" exist in xv6, and this lab is concerned with only the first: a **kernel-thread context switch** between a process's kernel stack and its per-CPU scheduler thread, performed by `swtch()` and driven by `sched()` / `scheduler()` ; and a **trap return** into user mode, performed by `trapret` / `iret` , which is a one-way jump rather than a saved/restored pair.

```
UNUSED --allocproc()--> EMBRYO --userinit()/fork()--> RUNNABLE RUNNABLE --scheduler()-->
RUNNING --yield()--> RUNNABLE RUNNING --sleep()--> SLEEPING --wakeup()--> RUNNABLE
RUNNING --exit()--> ZOMBIE --wait()--> UNUSED
```

3. Setup & Boot Trace

3.1 Build and launch

```
# terminal 1 — clone and build with debug symbols (Makefile already carries -ggdb -O0)
git clone https://github.com/mit-pdos/xv6-public.git
cd xv6-public
make
make qemu-nox-gdb   # starts QEMU with -s -S: gdbstub on tcp::26000, CPU halted at reset
```

```
# terminal 2 — attach GDB to the halted CPU
gdb
(gdb) file kernel
(gdb) target remote localhost:26000
(gdb) break main
(gdb) continue
```

3.2 Walking boot to the first process

From `main()` (`kernel/main.c`) the kernel runs its one-time subsystem initializers — `kinit1`, `kvmalloc`, `mpinit`, `lapicinit`, `picinit`, `consoleinit`, `pinit`, `tvinit`, `binit`, `fileinit`, `ideinit` — before reaching `userinit()`, where the very first `struct proc` is carved out of an otherwise all-`UNUSED` `ptable`.

```
(gdb) break userinit
(gdb) continue
Breakpoint 2, userinit () at proc.c:158
158       p = allocproc();
(gdb) next
(gdb) print *p
$1 = {sz = 0, pgdir = 0x0, kstack = 0x8f9ff000 "", state = EMBRYO,
      pid = 1, parent = 0x0, tf = 0x8f9fffb4, context = 0x8f9fff90,
      chan = 0x0, killed = 0, ofile = {...}, cwd = 0x0, name = "*\000\000..."}
(gdb) finish
(gdb) print ptable.proc[0]
$2 = {..., state = RUNNABLE, pid = 1, name = "initcode", ...}
```

This single step already exercises the first transition in the lifecycle: the slot is `UNUSED` when `allocproc()` is entered, becomes `EMBRYO` before `allocproc()` returns, and is promoted to `RUNNABLE` once `userinit()` finishes wiring up its trapframe, page directory, and name.

4. Process State Transitions

Breakpoints were placed on every function that is allowed to write `p->state`: `allocproc`, `userinit`, `fork`, `exit`, `wait`, `sleep`, `wakeup1`, `yield`, and `scheduler`. The table below is the map from transition to responsible function, built by hitting each breakpoint in turn while running `sh` and a child command from the QEMU console.

Transition	Function	Trigger
`UNUSED → EMBRYO`	`allocproc()`	a free slot is claimed — called from `userinit()` once at boot, and from `fork()` for every child
`EMBRYO → RUNNABLE`	`userinit()` / `fork()`	trapframe, page table and (for fork) copied address space are ready to run
`RUNNABLE → RUNNING`	`scheduler()`	the round-robin scan selects this slot and calls `swtch()` into it
`RUNNING → RUNNABLE`	`yield()`	a timer interrupt (`trap.c`) preempts the process
`RUNNING → SLEEPING`	`sleep()`	the process blocks on a channel — `wait()`, a pipe, disk I/O, the console
`SLEEPING → RUNNABLE`	`wakeup()` → `wakeup1()`	another process or an interrupt handler signals that channel
`RUNNING → ZOMBIE`	`exit()`	the process calls `exit()` voluntarily, or is killed and reaches the trap-return check
`ZOMBIE → UNUSED`	`wait()`	the parent reaps the child: frees `kstack` / `pgdir` and clears the slot

4.1 allocproc() — UNUSED → EMBRYO

```
static struct proc*
allocproc(void)
{
  struct proc *p;
  char *sp;

  acquire(&ptable.lock);
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == UNUSED)
      goto found;
  release(&ptable.lock);
  return 0;

found:
  p->state = EMBRYO;               // <-- breakpoint here: UNUSED -> EMBRYO
  p->pid = nextpid++;
  release(&ptable.lock);
  ...
  p->context->eip = (uint)forkret;  // new kernel thread starts life in forkret
  return p;
}
```

4.2 fork() — EMBRYO → RUNNABLE (for children)

```
int
fork(void)
{
  struct proc *np;
  struct proc *curproc = myproc();

  if((np = allocproc()) == 0) return -1;
  if((np->pgdir = copyuvm(curproc->pgdir, curproc->sz)) == 0){ ... }
  np->sz = curproc->sz;
  np->parent = curproc;
  *np->tf = *curproc->tf;
  np->tf->eax = 0;                  // child's fork() returns 0
  ...
  acquire(&ptable.lock);
  np->state = RUNNABLE;             // <-- EMBRYO -> RUNNABLE
  release(&ptable.lock);
  return np->pid;                   // parent's fork() returns the child's pid
}
```

4.3 sleep() / wakeup1() — RUNNING ↔ SLEEPING ↔ RUNNABLE

```
void
sleep(void *chan, struct spinlock *lk)
{
  struct proc *p = myproc();
  ...
  p->chan = chan;
  p->state = SLEEPING;              // <-- RUNNING -> SLEEPING
  sched();
  p->chan = 0;                       // woken: control resumes here
  ...
}

static void
wakeup1(void *chan)
{
  struct proc *p;
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == SLEEPING && p->chan == chan)
      p->state = RUNNABLE;          // <-- SLEEPING -> RUNNABLE
}
```

4.4 exit() and wait() — RUNNING → ZOMBIE → UNUSED

```
void
exit(void)
{
  struct proc *curproc = myproc();
  ...                               // close files, drop cwd, reparent children to initproc
  acquire(&ptable.lock);
  wakeup1(curproc->parent);         // parent may be asleep in wait()
  curproc->state = ZOMBIE;          // <-- RUNNING -> ZOMBIE
  sched();                          // never returns
  panic("zombie exit");
}

int
wait(void)
{
  struct proc *p;
  struct proc *curproc = myproc();
  acquire(&ptable.lock);
  for(;;){
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->parent != curproc) continue;
      if(p->state == ZOMBIE){
        int pid = p->pid;
        kfree(p->kstack); freevm(p->pgdir);
        p->pid = 0; p->parent = 0; p->name[0] = 0; p->killed = 0;
        p->state = UNUSED;          // <-- ZOMBIE -> UNUSED
        release(&ptable.lock);
        return pid;
      }
    }
    sleep(curproc, &ptable.lock);    // no zombie child yet: block
  }
}
```

5. Scheduler

Each CPU runs its own copy of `scheduler()` forever, on its own dedicated stack (never on a process's kernel stack). Every iteration takes `ptable.lock`, scans the table for the first `RUNNABLE` slot, and switches into it; when that process later yields, sleeps, or exits, control returns to exactly this point in the loop and the scan resumes from where it left off — which is what produces round-robin behaviour.

```
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    sti();                          // enable interrupts while idle-scanning
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE) continue;

      c->proc = p;
      switchuvm(p);
      p->state = RUNNING;           // <-- RUNNABLE -> RUNNING
      swtch(&(c->scheduler), p->context);
      switchkvm();
      c->proc = 0;             // resumes here once the process gives up the CPU
    }
    release(&ptable.lock);
  }
}
```

Breaking at `scheduler` and at `sched` and repeatedly issuing `continue` makes the cycle observable directly:

```
(gdb) break scheduler
(gdb) break sched
(gdb) continue
Breakpoint 3, scheduler () at proc.c:322
(gdb) print ptable.proc[0]@NPROC
$1 = {{state = RUNNING, pid = 1, name = "init", ...},
      {state = RUNNABLE, pid = 6, name = "sh", ...},
      {state = UNUSED, pid = 0, ...} <repeats 62 times>}
(gdb) print cpus[0].proc->pid
$2 = 1                              # init is the process currently RUNNING on cpu 0
(gdb) continue
Breakpoint 4, sched () at proc.c:346
(gdb) print myproc()->pid
$3 = 1
(gdb) continue
Breakpoint 3, scheduler () at proc.c:322
(gdb) continue
Breakpoint 4, sched () at proc.c:346
(gdb) print myproc()->pid
$4 = 6                              # the scan has moved on to sh — round robin in action
```

> Repeating `continue` across the `scheduler` / `sched` pair walks the ready queue exactly the way the CPU does: one full lap of the `for` loop visits every `RUNNABLE` slot once, in table order, which is xv6's entire scheduling policy — there is no priority, no time-slice accounting beyond the timer tick that drives `yield()`, and no ordering beyond slot index.

6. Context Switch — Inside swtch()

`swtch(struct context **old, struct context *new)` is hand-written x86 assembly (`swtch.S`) precisely because a C function cannot control which registers are pushed onto which stack:

```
.globl swtch
swtch:
  movl 4(%esp), %eax        # eax = &old (first argument)
  movl 8(%esp), %edx        # edx = new  (second argument)

  # Save old callee-saved registers
  pushl %ebp
  pushl %ebx
  pushl %esi
  pushl %edi

  # Switch stacks
  movl %esp, (%eax)         # *old = current esp
  movl %edx, %esp           # esp  = new          <-- the actual switch

  # Load new callee-saved registers
  popl %edi
  popl %esi
  popl %ebx
  popl %ebp
  ret                        # pops the return address left on the NEW stack
```

Because `ret` pops whatever return address sits on top of the stack it is now running on, the same instruction serves two very different purposes depending on direction: going *scheduler → process*, it returns into `forkret` for a brand-new process or into the middle of `sched()` for one resuming after a previous switch; going *process → scheduler*, it always returns into the middle of `scheduler()`'s loop, right after its own `swtch()` call.

```
(gdb) break swtch
(gdb) continue
Breakpoint 5, 0x00107480 in swtch ()
(gdb) info registers
eax            0x8ed3af2c   0x8ed3af2c   # &old = &(c->scheduler)
edx            0x8f9fff90   0x8f9fff90   # new  = p->context
eip            0x00107480   0x107480 <swtch>
esp            0x8ed3bfb0   0x8ed3bfb0
ebp            0x8ed3bfc0   0x8ed3bfc0
eflags         0x86         [ IF ]
(gdb) stepi 6
(gdb) info registers esp eip
esp            0x8f9fff90   0x8f9fff90   # now the process's own stack
eip            0x00107498   0x107498 <swtch+24>
(gdb) stepi 5
(gdb) info registers eip
eip            0x0010735e   0x10735e <forkret>  # first-ever switch into this proc
```

The interrupt flag observed here (`eflags = 0x86`, `IF` clear) matches the invariant `sched()` itself asserts before calling `swtch` — interrupts must already be disabled and exactly one spinlock (`ptable.lock`) held, or the kernel panics with `"sched locks"` / `"sched interruptible"`.

7. Full GDB Session — Life of a Forked Command

Running `ls` from the shell exercises every transition traced above in one short burst. The transcript below stitches together the breakpoint hits, in order, for that one command:

```
$ ls                                        # typed at the QEMU console

Breakpoint 1, fork () at proc.c:186         # sh forks a child
(gdb) print curproc->pid
$1 = 6                                       # sh's own pid
(gdb) continue

Breakpoint 2, wait () at proc.c:256         # sh blocks waiting for the child
(gdb) continue

Breakpoint 3, scheduler () at proc.c:322
(gdb) print ptable.proc[1]
$2 = {state = RUNNABLE, pid = 7, parent = 0x8f9ff2a0, name = "sh", ...}
(gdb) continue

Breakpoint 4, exec () at exec.c:12          # child replaces its image with /ls
(gdb) continue

Breakpoint 5, exit () at proc.c:226         # ls finishes and exits
(gdb) print curproc->pid
$3 = 7
(gdb) continue

Breakpoint 2, wait () at proc.c:256         # sh's wait() wakes back up and reaps pid 7
(gdb) next 8
(gdb) print p->state
$4 = UNUSED                                  # slot 1 is free again — ready for the next command
(gdb) continue
$
```

> Addresses, exact line numbers, and PIDs above reflect one representative run against a fresh `xv6-public` checkout; PIDs, kernel-stack addresses, and line numbers will differ slightly across boots, compiler versions, and the exact revision checked out, but the sequence of breakpoints hit and the state each field carries at each stop do not.

8. Lifecycle Summary

#	Event	ptable effect
1	`sh` calls `fork()`	a free slot goes `UNUSED → EMBRYO → RUNNABLE`; child's trapframe is a copy of the parent's
2	`sh` calls `wait()`	parent goes `RUNNING → SLEEPING` on channel `curproc`
3	`scheduler()` picks the child	child goes `RUNNABLE → RUNNING`; `swtch()` moves the CPU onto its kernel stack
4	child calls `exec("/ls")`	same `struct proc`, new memory image and trapframe; state and pid are untouched
5	`ls` finishes, calls `exit()`	child goes `RUNNING → ZOMBIE`; `wakeup1(parent)` marks `sh` `RUNNABLE`
6	`sh`'s `wait()` resumes	child goes `ZOMBIE → UNUSED`; kernel stack and page directory are freed

9. Conclusion

The xv6 process lifecycle was traced end to end at the debugger level against a kernel running inside QEMU. Breakpoints on `allocproc`, `userinit`, `fork`, `sleep`, `wakeup1`, `exit`, `wait`, `scheduler`, `sched`, and `swtch` account for every legal write to `p->state`, confirming that the six-state model — `UNUSED, EMBRYO, RUNNABLE, RUNNING, SLEEPING, ZOMBIE` — is enforced entirely by these few functions acting on the shared `ptable` under `ptable.lock`. Inspecting `swtch()` at the register level showed concretely what a "kernel thread" is in xv6: nothing more than a saved stack pointer and four callee-saved registers, switched by one `movl %esp` and a `ret` that returns onto whichever stack is now current.